

these packages being installed if not already present. Now that our software is installed and updated, let's get the kernel source code:

```
- name: Updating Linux Kernel source code
  git: repo=https://github.com/torvalds/linux
  dest={{ code_path }}
```

This task is simple enough. We are using the `git` command, and providing it a repository location and a destination to place the source code. The magic of this task is that not only will it clone the repository at the supplied destination the first time it is run, but it will also keep it up-to-date during subsequent runs.

Note how we're using the `code_path` variable to supply the code path instead of writing that path all over again. Putting a variable name in double curly braces will tell Ansible to place its value there. We could likewise create a variable that points to the code repository, which would allow easy switching between GitHub and Kernel.org or yet another location.

Now we can combine all the rest of the build commands into one task:

```
- name: Compiling kernel
  shell: make mrproper && make def-
  config && make -j4 && tarxz-pkg
  chdir={{ code_path }}
```

This will launch the build process in a shell.

Unfortunately, the above command can be quite wasteful. If we add the above command to our tasks it will run each time, and end up recompiling the kernel even if there have been no updates. It would be better if we could just run this task if the kernel has been updated.

As it turns out, this is quite simple! All you need to do is to move the compilation task out of the tasks list and into another section called `handlers`. Then the `git` command can be configured to notify the compile command, so it will only run if there is an update. Here you can see the relevant bits of code:

```
---
- hosts: localhost
  tasks:
  ### Other tasks...
  - name: Updating Linux Kernel source code
```

```
  git: repo=https://github.com/torvalds/linux
  dest={{ code_path }}
  notify:
  - Compiling kernel
  handlers:
  - name: Compiling kernel
    shell: make mrproper && make def-
    config && make -j4 && make tarxz-pkg
    chdir={{ code_path }}
  ...
```

Now that the playbook is complete, you can run it with Ansible as follows:

```
ansible-playbook build-kernel.yaml -i hosts -c local
```

Here `-i hosts` tells the `ansible-playbook` command to use the inventory of hosts in a file called `hosts` in the directory in which the command is being run. You may wish to change this if your inventory file is elsewhere or named differently. The `-c local` bit of the command is to be used if you want to run the commands on your own local machine rather than over SSH. Remember to omit that if you are using a remote server.

Here we've only looked at the `apt`, `git` and `shell` modules. Ansible has hundreds of modules for everything from creating users, to sending SMS notifications using Twilio. You can find out about all the command modules included with Ansible here <http://dgit.in/AnsibleDoc>.

Conclusion

Using any configuration management tool at all is a HUGE advantage over using none. Unlike some other tools it does not need to be installed on the remote server. All it needs on the remote server is Python 2, and in some cases not even that. It connects to the remote server using simple SSH and transfers scripts that run your tasks.

The initial process of learning how to create playbooks and getting everything just right can be daunting, but once you have a working playbook, setting up a server with everything up and running becomes a matter of executing a single command.>

You can find out more about Ansible at <http://ansible.com>

*Links Inventory



*Semaphore

>>> Semaphore is a free and open-source alternative to Ansible Tower, the commercial and expensive web UI for Ansible.

dgit.in/semaphore

ANSIBLE TOWER

/ SYSTEM TRACKING
/ REMOTE COMMAND EXECUTION
/ SUPPORT FOR OPENSTACK
/ STREAMLINED UI

*YAML Syntax

>>> If you want a quick introduction to YAML in the context of Ansible, you can find it in Ansible's own documentation here.

dgit.in/YAMLSyntax



*Vagrant and Ansible

>>> If you use Vagrant then you'll be glad to know that Vagrant and Ansible work well together. Check the documentation here.

dgit.in/AnsVagrant